Natural Language Processing (NLP) - Overview

Natural Language Processing (NLP) is a field that combines computer science, artificial intelligence and language studies. It helps computers understand, process and create human language in a way that makes sense and is useful. With the growing amount of text data from social media, websites and other sources, NLP is becoming a key tool to gain insights and automate tasks like analyzing text or translating languages.

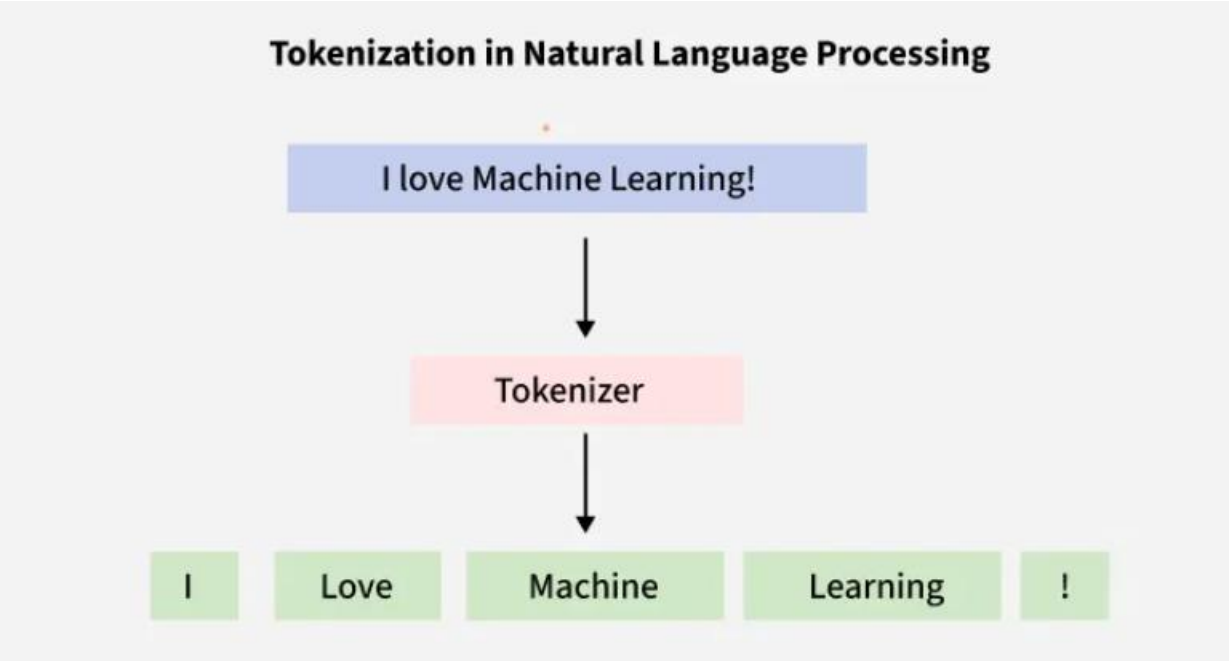
NLP is used by many applications that use language, such as text translation, voice recognition, text summarization and chatbots. You may have used some of these applications yourself, such as voice-operated GPS systems, digital assistants, speech-to-text software and customer service bots. NLP also helps businesses improve their efficiency, productivity and performance by simplifying complex tasks that involve language.

NLP Techniques:

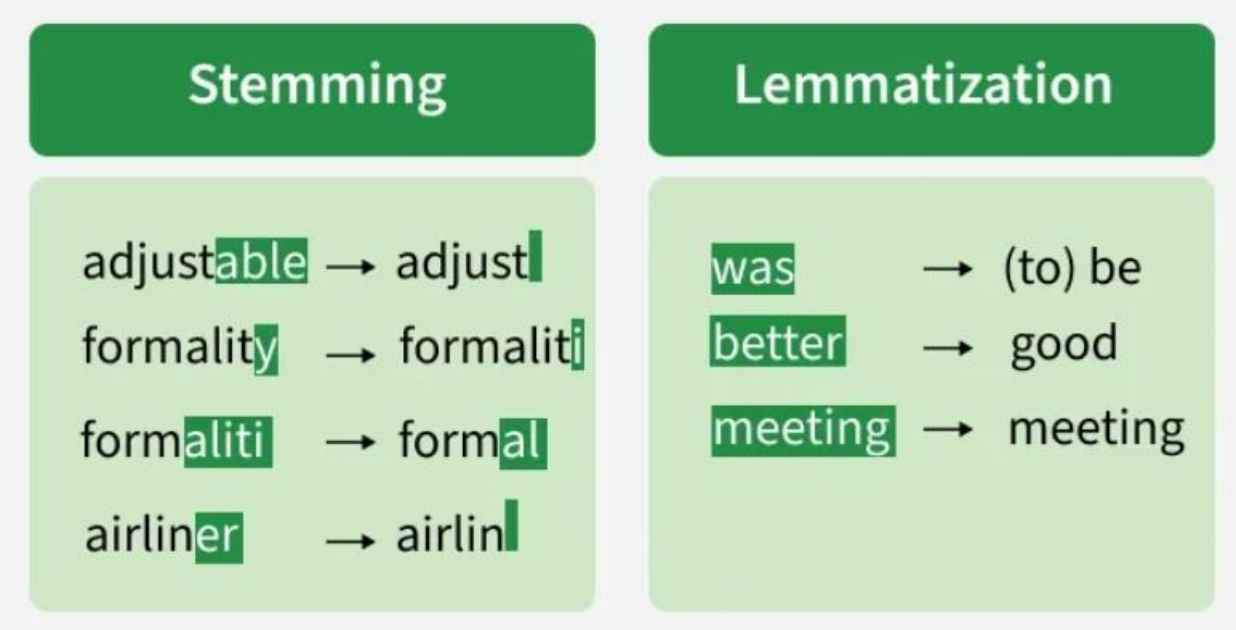
NLP encompasses a wide array of techniques that aimed at enabling computers to process and understand human language. These tasks can be categorized into several broad areas, each addressing different aspects of language processing. Here are some of the key NLP techniques:

Q. **Explain the major steps involved in NLP preprocessing. Illustrate with suitable examples.**

1. Text Processing and Preprocessing

- **Tokenization:** Dividing text into smaller units, such as words or sentences.
- 

- **Stemming and Lemmatization**: Reducing words to their base or root forms.
  Lemmatization and stemming are two popular text-preprocessing techniques in NLP used to reduce words to their base form. While stemming cuts words down to their root by trimming endings, lemmatization uses linguistic rules to return meaningful base words (lemmas). Understanding the difference helps improve text analysis, search accuracy and NLP model performance.



-
- **Stopword Removal**: Removing common words (like "and", "the", "is") that may not carry significant meaning.
- Natural language processing tasks often involve filtering out commonly occurring words that provide no or very little semantic value to text analysis. These words are known as stopwords include articles, prepositions and pronouns like "the", "and", "is" and "in". While they seem insignificant, proper stopword handling can dramatically impact the performance and accuracy of NLP applications.

Sample text with Stop Words	Without Stop Words
GeeksforGeeks – A Computer Science Portal for Geeks	GeeksforGeeks, Computer Science, Portal, Geeks
Can listening be exhausting?	Listening, Exhausting
I like reading, so I read	Like, Reading, read

- **Text Normalization:** Standardizing text, including case normalization, removing punctuation and correcting spelling errors.

Steps Required

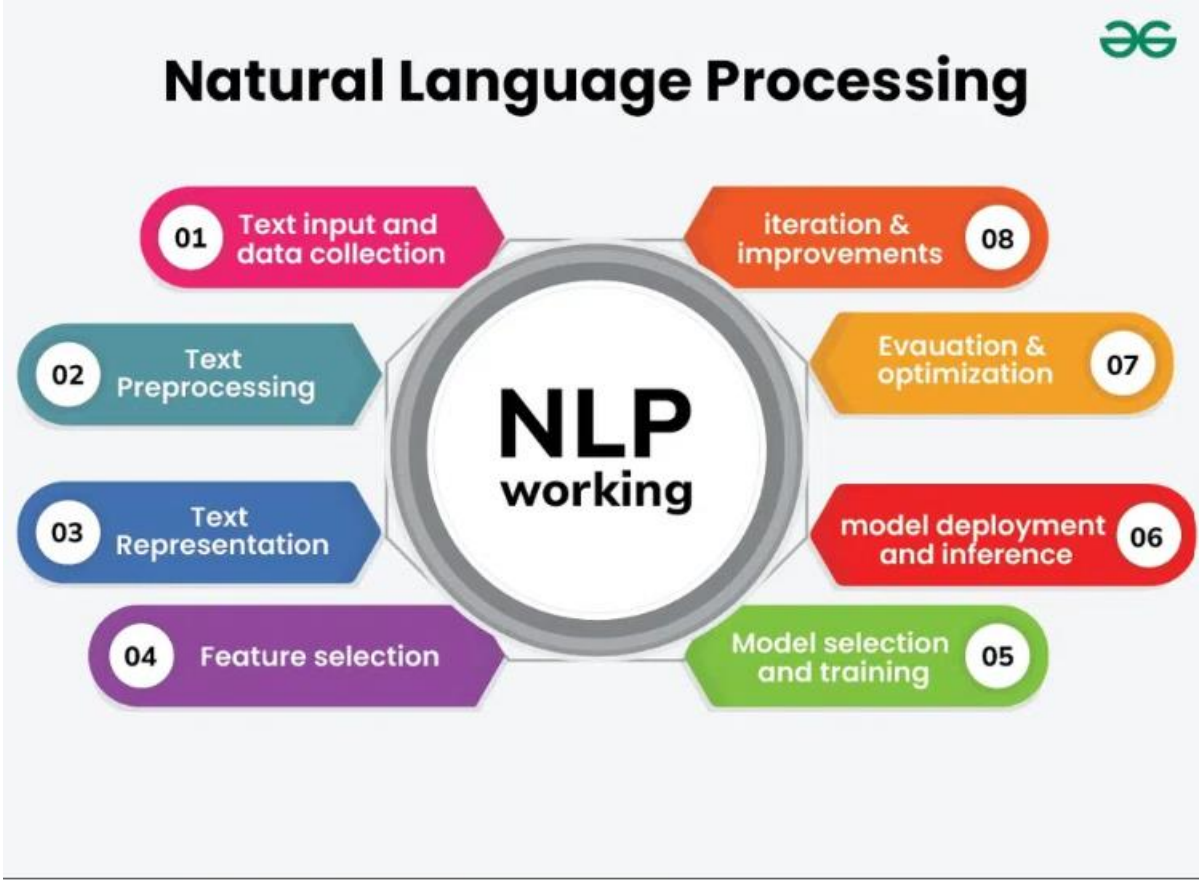
Here, we will discuss some basic steps need for Text normalization.

- Input text String,
- Convert all letters of the string to one case(either lower or upper case),
- If numbers are essential to convert to words else remove all numbers,
- Remove punctuations, other formalities of grammar,
- Remove white spaces,
- Remove stop words,
- And any other computations.

2. Text Classification in NLP

- **Sentiment Analysis:** Determining the sentiment or emotional tone expressed in a text (e.g., positive, negative, neutral).
- **Topic Modeling:** Identifying topics or themes within a large collection of documents.
- **Spam Detection:** Classifying text as spam or not spam.

How Natural Language Processing (NLP) Works:



1. Text Input and Data Collection

- **Data Collection:** Gathering text data from various sources such as websites, books, social media or proprietary databases.
- **Data Storage:** Storing the collected text data in a structured format, such as a database or a collection of documents.

2. Text Preprocessing

Preprocessing is crucial to clean and prepare the raw text data for analysis. Common preprocessing steps include:

- **Tokenization:** Splitting text into smaller units like words or sentences.

- **Lowercasing:** Converting all text to lowercase to ensure uniformity.
- **Stopword Removal:** Removing common words that do not contribute significant meaning, such as "and," "the," "is."
- **Punctuation Removal:** Removing punctuation marks.
- **Stemming and Lemmatization:** Reducing words to their base or root forms. Stemming cuts off suffixes, while lemmatization considers the context and converts words to their meaningful base form.
- **Text Normalization:** Standardizing text format, including correcting spelling errors, expanding contractions and handling special characters.

3. Text Representation

- **Bag of Words (BoW):** Representing text as a collection of words, ignoring grammar and word order but keeping track of word frequency.

Q.What is Bag of Words (BoW)?

The Bag of Words (BoW) model is the simplest form of text representation in numbers. Like the term itself, we can represent a sentence as a bag of words vector (a string of numbers).

Let's recall the three types of movie reviews we saw earlier:

- **Review 1:** This movie is very scary and long
- **Review 2:** This movie is not scary and is slow
- **Review 3:** This movie is spooky and good

We will first build a vocabulary from all the unique words in the above three reviews. The vocabulary consists of these 11 words: ‘This’, ‘movie’, ‘is’, ‘very’, ‘scary’, ‘and’, ‘long’, ‘not’, ‘slow’, ‘spooky’, ‘good’.

We can now take each of these words and mark their occurrence in the three movie reviews above with 1s and 0s. This will give us 3 vectors for 3 reviews:

	1 This	2 movie	3 is	4 very	5 scary	6 and	7 long	8 not	9 slow	10 spooky	11 good	Length of the review(in words)
Review 1	1	1	1	1	1	1	1	0	0	0	0	7
Review 2	1	1	2	0	0	1	1	0	1	0	0	8
Review 3	1	1	1	0	0	0	1	0	0	1	1	6

Vector-of-Review1: [1 1 1 1 1 1 1 0 0 0 0]

Vector-of-Review2: [1 1 2 0 0 1 1 0 1 0 0]

Vector-of-Review3: [1 1 1 0 0 0 1 0 0 1 1]

And that’s the core idea behind a Bag of Words (BoW) model.

Drawbacks of using a Bag-of-Words (BoW) Model

In the above example, we can have vectors of length 11. However, we start facing issues of Bag of Words when we come across new sentences:

1. If the new sentences contain new words, then our vocabulary size would increase and thereby, the length of the vectors would increase too.

2. Additionally, the vectors would also contain many 0s, thereby resulting in a sparse matrix (which is what we would like to avoid)
3. We are retaining no information on the grammar of the sentences nor on the ordering of the words in the text.

- **Term Frequency-Inverse Document Frequency (TF-IDF):** A statistic that reflects the importance of a word in a document relative to a collection of documents.

What is TF-IDF?

TF-IDF is a numerical statistic that reflects the significance of a word within a document relative to a collection of documents, known as a corpus. The idea behind TF-IDF is to quantify the importance of a term in a document with respect to its frequency in the document and its rarity across multiple documents.

Term Frequency (TF)

TF measures the frequency of a term within a document. It is calculated as the ratio of the number of times a term occurs in a document to the total number of terms in that document. The goal is to emphasize words that are frequent within a document.

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

Inverse Document Frequency (IDF)

IDF measures the rarity of a term across a collection of documents. It is calculated as the logarithm of the ratio of the total number of documents to the number of documents containing the term. The goal is to penalize words that are common across all documents.

$$IDF(t, D) = \log\left(\frac{\text{Total number of documents in the corpus } N}{\text{Number of documents containing term } t}\right)$$

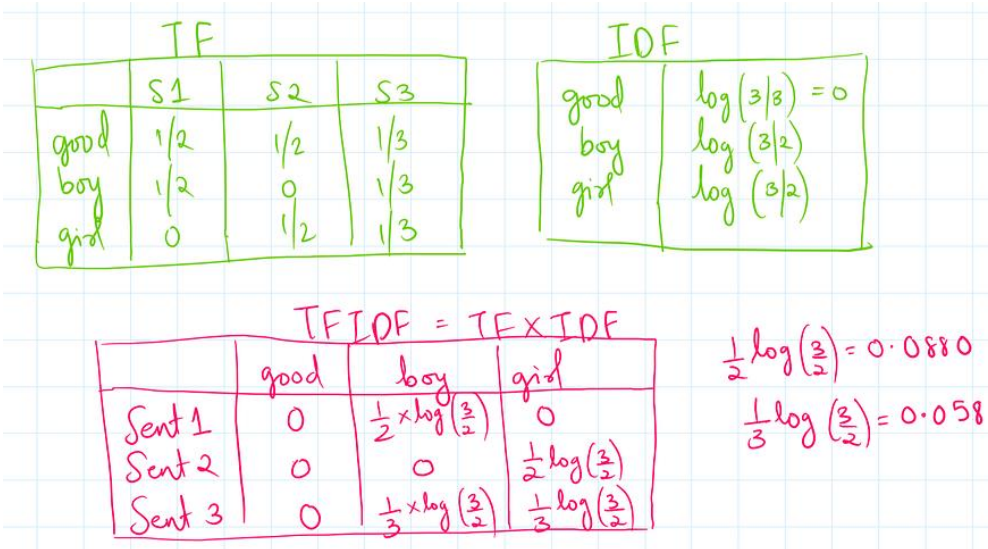
Combining TF and IDF: TF-IDF

The TF-IDF score for a term in a document is obtained by multiplying its TF and IDF scores.

Sent 1 : good boy
Sent 2 : good girl
Sent 3 : boy girl good

$$TF = \frac{\text{No of rep of words in a sentence}}{\text{No of words in a sentence}}$$

$$IDF = \log\left(\frac{\text{No of sentences}}{\text{No of sentences containing word}}\right)$$



- **Word Embeddings**: Using dense vector representations of words where semantically similar words are closer together in the vector space (e.g., Word2Vec, GloVe).

Q. **What are Word Embeddings?**

✅ Word Embedding

🔷 Definition

Word Embedding is a technique in **Natural Language Processing (NLP)** that converts words into **dense numerical vectors** such that words with similar meanings have similar vector representations.

In simple words:

Word Embedding = Representing words as meaningful numbers.

🔷 Why Do We Need Word Embeddings?

Q.Describe Word2Vec models — Continuous Bag of Words (CBOW) and Skip-Gram. Explain their working with examples.

Computers cannot understand text directly.
They only understand numbers.

Traditional methods like:

- One-Hot Encoding
- Bag of Words

❌ Problems:

- Very high dimensional
- Sparse vectors
- No semantic meaning captured

Word Embeddings solve this problem by:

- Reducing dimensionality
- Capturing semantic relationships
- Representing similar words closely in vector space

🔷 Example

Instead of:

Dog → [0,0,1,0,0,0]

Word embedding gives:

Dog → [0.21, 0.78, -0.44, 0.11]

Cat → [0.19, 0.75, -0.40, 0.10]

Notice:
Dog and Cat vectors are similar because they are semantically related.

🔷 Key Idea

Words appearing in similar contexts have similar meanings.

This is called:

Distributional Hypothesis

Example:

- "King" and "Queen" appear in similar contexts.
- Their vectors will be close.

🔷 Types of Word Embeddings

1️⃣ Word2Vec

Developed by Google.

Two models:

- CBOW (Continuous Bag of Words)
- Skip-Gram

✔ Learns embeddings using neural networks.

2️⃣ GloVe (Global Vectors)

Developed by Stanford.

✔ Uses global word co-occurrence matrix
✔ Combines statistical and neural methods

3️⃣ FastText

Developed by Facebook.

✓ Considers sub-words
✓ Handles rare words better

🔷 Mathematical Representation

Each word is mapped to a vector in $\mathbb{R}^n$ space.

Example:

If embedding size = 100
Each word → 100-dimensional vector

🔷 Properties of Word Embeddings

✓ Dense vectors
✓ Low dimensional
✓ Capture semantic meaning
✓ Can perform vector arithmetic

Example:

King − Man + Woman ≈ Queen

🔷 Advantages

- Captures semantic similarity
- Reduces dimensionality
- Improves NLP model performance
- Works well in deep learning models

🔷 Limitations

- Static embeddings (Word2Vec, GloVe) give one meaning per word
  Example: “Bank” (river bank vs financial bank)
- Requires large data to train

◆ Applications

- Machine Translation
- Sentiment Analysis
- Chatbots
- Text Classification
- Question Answering
- Speech Recognition

4. Feature Extraction

Extracting meaningful features from the text data that can be used for various NLP tasks.

- **N-grams:** Capturing sequences of N words to preserve some context and word order.
- **Syntactic Features**: Using parts of speech tags, syntactic dependencies and parse trees.
- **Semantic Features**: Leveraging word embeddings and other representations to capture word meaning and context.

5. Model Selection and Training

Selecting and training a machine learning or deep learning model to perform specific NLP tasks.

- Supervised Learning**:** Using labeled data to train models like Support Vector Machines (SVM), Random Forests or deep learning models like Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs).
- Unsupervised Learning**: Applying** techniques like clustering or topic modeling (e.g., Latent Dirichlet Allocation) on unlabeled data.

- Pre-trained Models: **Utilizing** pre-trained language models such as BERT, GPT or transformer-based models that have been trained on large corpora.

Q.6 Consider the sentence:

"Deep learning is powerful"

Using **CBOW model** with window size = 1:

1. Construct the vocabulary.
2. Write the one-hot encoding for the word **"learning"**.
3. If the embedding size is 2 and the weight matrix W is given as:

$$W = \begin{bmatrix} 0.1 & 0.3 \\ 0.4 & 0.2 \\ 0.6 & 0.5 \\ 0.7 & 0.8 \end{bmatrix}$$

Compute the hidden layer vector for the word **"learning"**.

Solution:

Sentence:
"Deep learning is powerful"

Window size = 1 (CBOW)

✅ 1️⃣ Construct the Vocabulary

Words in the sentence:

1. Deep
2. learning

3. is
4. powerful

So vocabulary size = **4**

We assign index positions:

Word	Index
Deep	0
learning	1
is	2
powerful	3

✅ 2️⃣ One-Hot Encoding for "learning"

Vocabulary size = 4

One-hot vector length = 4

Since **learning is index 1**:

learning=[0 1 0 0]

✅ 3️⃣ Compute Hidden Layer Vector

Given embedding size = 2

Weight matrix W is:

$$W = \begin{bmatrix} 0.1 & 0.3 \\ 0.4 & 0.2 \\ 0.6 & 0.5 \\ 0.7 & 0.8 \end{bmatrix}$$

Matrix dimension = 4 × 2
(Vocabulary size × Embedding size)

◈ Hidden Layer Computation (CBOW)

Hidden layer vector is computed as:

$$h = xWh$$

Where:

x = one-hot vector

W = weight matrix

Multiply:

$$[0\ 1\ 0\ 0] \begin{bmatrix} 0.1 & 0.3 \\ 0.4 & 0.2 \\ 0.6 & 0.5 \\ 0.7 & 0.8 \end{bmatrix}$$

Since one-hot has 1 at position 1,
we simply select **row 2** (index 1):

$$h = [0.4\ \ 0.2]$$

✅ Final Answers

1 Vocabulary

{Deep, learning, is, powerful}

2 One-hot encoding of “learning”

[0 1 0 0][0 \; 1 \; 0 \; 0][0100]

3 Hidden layer vector

h=[0.4 0.2]h = [0.4 \; 0.2]h=[0.40.2]